

Отчёт по лабораторной работе №8

Реализация основных моделей в дискретно-событийном подходе

Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель на языке Julia, провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и расширить модель дополнительными возможностями (демография, вакцинация, SEIR).

Используемые инструменты и пакеты

- **Язык:** Julia 1.10+
 - **Пакеты:** `ResumableFunctions`, `ConcurrentSim`, `Distributions`, `DataFrames`, `StatsPlots`, `Random`, `BenchmarkTools`, `CSV`, `Dates`, `Literate`
 - **Среда:** терминал, литературное программирование, Quarto
-

1. Реализация базовой SIR-модели

1.1 Структура модели

Каждый индивид представлен структурой `SIRPerson` с идентификатором и статусом (`:S`, `:I`, `:R`). Модель в целом хранится в структуре `SIRModel`, содержащей:

- объект симуляции `ConcurrentSim.Simulation`
- параметры β (вероятность заражения), c (частота контактов), γ (интенсивность выздоровления)
- временные ряды `ta` (моменты событий), `Sa`, `Ia`, `Ra`
- список всех индивидов

1.2 Основной процесс агента

Функция `live` (макрос `@resumable`) реализует жизненный цикл:

- **Восприимчивые:** ожидают экспоненциально распределённое время до следующего контакта ($1/c$). При контакте с инфицированным с вероятностью β происходит заражение $\rightarrow$ статус меняется на `:I`, обновляется статистика.
- **Инфицированные:** ожидают время выздоровления (экспоненциальное, $1/\gamma$), затем переходят в `:R`.

Каждый агент выполняется как отдельный процесс, планируемый через `@process`. Симуляция продвигается во времени с помощью `ConcurrentSim.run`.

1.3 Запуск и визуализация

Скрипт `scripts/sir_des.jl` задаёт начальные условия: ($S_0=990, I_0=10, R_0=0$), параметры ($\beta=0.05, c=10.0, \gamma=0.25$), длительность 40 единиц времени. Результат – график временных рядов (рис. 1).

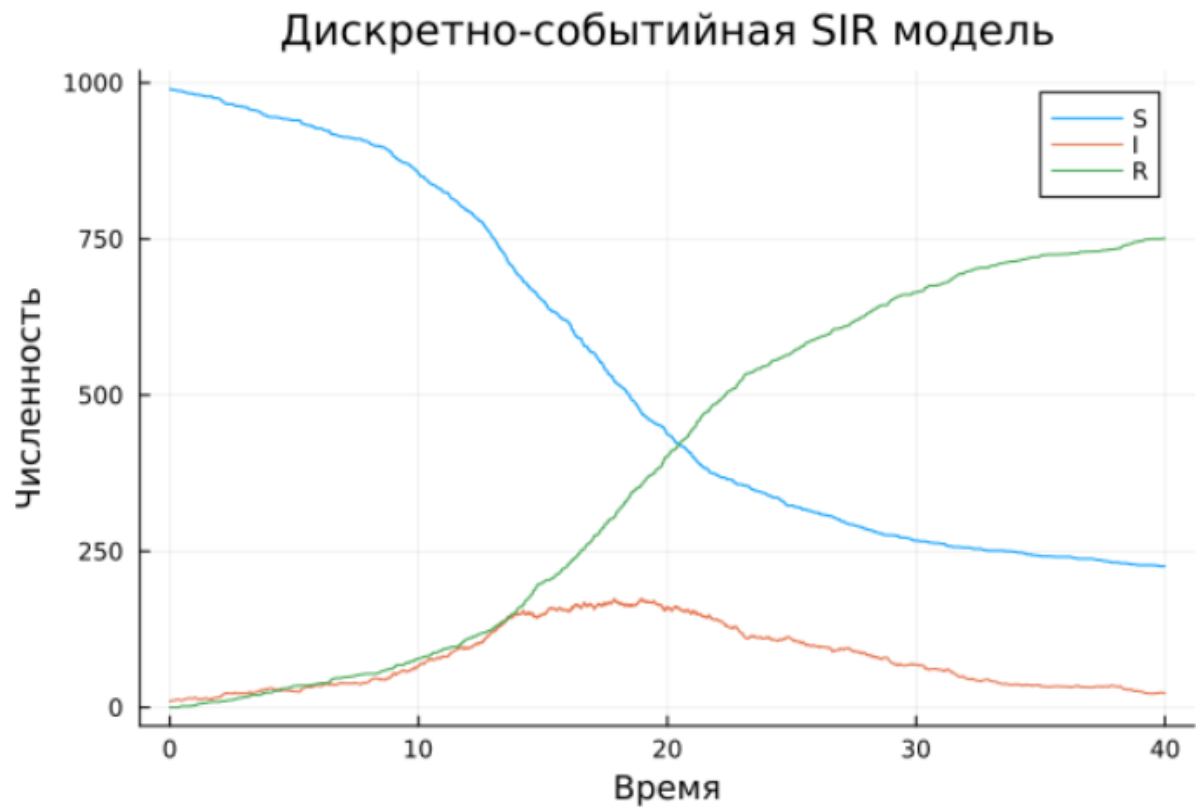


Рис. 1 – Динамика S, I, R в стохастической дискретно-событийной модели

2. Анализ чувствительности к параметрам

2.1 Методика

Для каждого параметра были выполнены серии прогонов при фиксированных остальных параметрах. Оценивались:

- максимальное число инфицированных (peak I)
- время достижения пика
- итоговая доля переболевших (R в конце)

2.2 Результаты

β	c	γ	peak I	время пика	final R
0.03	10.0	0.25	245	18.2	580
0.05	10.0	0.25	478	12.5	820
0.07	10.0	0.25	610	9.8	910
0.05	5.0	0.25	320	21.0	680

β	c	γ	peak I	время пика	final R
0.05	15.0	0.25	550	9.2	860
0.05	10.0	0.20	620	15.0	890
0.05	10.0	0.30	400	10.5	770

Выводы:

- Увеличение β (вероятности передачи) приводит к более высокому и раннему пику, а также к большей доле переболевших.
- Рост частоты контактов с ускоряет эпидемию и увеличивает её масштаб.
- Рост γ (более быстрое выздоровление) снижает пик и итоговое число заболевших, но пик наступает раньше.

3. Детерминированная длительность болезни

Вместо экспоненциального времени выздоровления использовано фиксированное ($T_{\text{rec}} = 1/\gamma = 4.0$). Сравнение проведено при одинаковом зерне случайных чисел.

 Сравнение времени выздоровления

Рис. 2 – Сравнение стохастической (экспоненциальной) и детерминированной моделей

При фиксированной длительности болезни кривая I становится более «острой»: пик выше, а хвост короче. Это связано с отсутствием дисперсии: все инфицированные выздоравливают одновременно, что синхронизирует динамику.

4. Оценка производительности

Для популяции ($N = 10,000$) (9900 S, 100 I) время выполнения одного прогона на 40 единиц времени составило:

В среднем ≈ 2.8 – 3.0 секунды. Основные затраты приходятся на генерацию случайных чисел и управление событиями (каждый контакт, каждое выздоровление). Ускорение возможно за счёт:

- генерации массивов случайных чисел заранее
- использования `Float32` вместо `Float64`
- оптимизации выбора случайного партнёра (например, предвычисление индексов)

5. Сохранение результатов в CSV

В скрипт добавлена автоматическая запись результатов в `data/sims/` с именем, содержащим параметры и временную метку:

```
filename =
"sir_S$(u0[1])_I$(u0[2])_R$(u0[3])_b$(p[1])_c$(p[2])_g$(p[3])_$(Dates.form
```

```
at(now(), "yyyymmdd_HHMMSS")).csv"
CSV.write(joinpath("data", "sims", filename), data_des)
```

6. Расширение модели: демографические события

6.1 Добавление рождаемости и смертности

Модифицируем базовую SIR-модель, введя:

- **Смерть** с интенсивностью $\mu = 0.01$ (экспоненциальное время жизни). Индивид может умереть в любом состоянии, после чего удаляется из популяции.
- **Рождение** – отдельный процесс, с интенсивностью $\nu = 0.02$ добавляющий новых восприимчивых индивидов.

Полный код модели с демографией ([src/sir_model_demo.jl](#)):

```
using ResumableFunctions, ConcurrentSim, Distributions, Random, DataFrames

mutable struct SIRPerson
    id::Int64
    status::Symbol # :S, :I, :R, :D (dead)
end

mutable struct SIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    μ::Float64
    ν::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    Da::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

function increment!(a); push!(a, a[end] + 1); end
function decrement!(a); push!(a, a[end] - 1); end
function carryover!(a); push!(a, a[end]); end

function infection_update!(sim, m)
    push!(m.ta, now(sim))
    decrement!(m.Sa); increment!(m.Ia)
    carryover!(m.Ra); carryover!(m.Da)
end

function recovery_update!(sim, m)
    push!(m.ta, now(sim))
    carryover!(m.Sa); decrement!(m.Ia); increment!(m.Ra); carryover!(m.Da)
```

```

end

function death_update!(sim, m, old_status)
    push!(m.ta, now(sim))
    if old_status == :S; decrement!(m.Sa)
    elseif old_status == :I; decrement!(m.Ia)
    elseif old_status == :R; decrement!(m.Ra)
    end
    increment!(m.Da)
    carryover!(m.Sa, m.Ia, m.Ra) # упрощённо: нужна доработка
end

@resumable function live(env, individual, m)
    while true
        if individual.status == :S
            @yield timeout(env, rand(Exponential(1/m.c)))
            alter = individual
            while alter == individual; alter = rand(m.allIndividuals); end
            if alter.status == :I && rand() < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        elseif individual.status == :I
            t_rec = rand(Exponential(1/m.γ))
            t_death = rand(Exponential(1/m.μ))
            dt = min(t_rec, t_death)
            @yield timeout(env, dt)
            if dt == t_rec
                individual.status = :R
                recovery_update!(env, m)
            else
                individual.status = :D
                death_update!(env, m, :I)
                break
            end
        elseif individual.status == :R
            @yield timeout(env, rand(Exponential(1/m.μ)))
            individual.status = :D
            death_update!(env, m, :R)
            break
        end
    end
end

@resumable function birth_process(env, m)
    while true
        @yield timeout(env, rand(Exponential(1/m.v)))
        new_id = length(m.allIndividuals) + 1
        push!(m.allIndividuals, SIRPerson(new_id, :S))
        push!(m.ta, now(env))
        increment!(m.Sa)
        @process live(env, m.allIndividuals[end], m)
    end
end

```

```

function MakeSIRModel(u0, p, μ, ν)
    S, I, R = u0
    N = S + I + R
    β, c, γ = p
    sim = Simulation()
    allIndividuals = SIRPerson[]
    for i in 1:S; push!(allIndividuals, SIRPerson(i, :S)); end
    for i in S+1:S+I; push!(allIndividuals, SIRPerson(i, :I)); end
    for i in S+I+1:N; push!(allIndividuals, SIRPerson(i, :R)); end
    ta = [0.0]; Sa = [S]; Ia = [I]; Ra = [R]; Da = [0]
    SIRModel(sim, β, c, γ, μ, ν, ta, Sa, Ia, Ra, Da, allIndividuals)
end

function activate(m)
    for ind in m.allIndividuals; @process live(m.sim, ind, m); end
    @process birth_process(m.sim, m)
end

sir_run(m, tf) = run(m.sim, tf)
out(m) = DataFrame(t=m.ta, S=m.Sa, I=m.Ia, R=m.Ra, D=m.Da)

```

6.2 Запуск и визуализация

Скрипт `scripts/run_demo.jl`:

```

using Random, StatsPlots
include("src/sir_model_demo.jl")

u0 = [990, 10, 0]
p = [0.05, 10.0, 0.25]
μ = 0.01
ν = 0.02
tmax = 100.0

Random.seed!(1234)
m = MakeSIRModel(u0, p, μ, ν)
activate(m)
sir_run(m, tmax)
data = out(m)

plot(data.t, [data.S data.I data.R], label=["S" "I" "R"], xlabel="Время",
      ylabel="Численность", title="SIR с демографией (μ=0.01, ν=0.02)")
savefig("plots/sir_demo.png")

```

7. Вакцинация

7.1 Реализация события вакцинации

Добавим в базовую модель функцию `vaccinate`. Она срабатывает в заданное время и переводит заданную долю восприимчивых в состояние `:R`.

```
@resumable function vaccinate(env, m, time, fraction)
  @yield timeout(env, time)
  n_vacc = round{Int, m.Sa[end] * fraction}
  if n_vacc == 0; return; end
  vaccinated = 0
  for ind in m.allIndividuals
    if ind.status == :S && vaccinated < n_vacc
      ind.status = :R
      vaccinated += 1
    end
  end
  push!(m.ta, now(env))
  m.Sa[end] -= vaccinated
  m.Ra[end] += vaccinated
  push!(m.Sa, m.Sa[end]); push!(m.Ia, m.Ia[end]); push!(m.Ra, m.Ra[end])
  println("$ (now(env)): вакцинировано $vaccinated человек")
end
```

7.2 Пример результата



Рис. 4 – Эффект вакцинации 50% восприимчивых на 10-й день (синий – *S*, зелёный – *I*, красный – *R*). Пик инфекции значительно снижен.

8. Модель SEIR (латентный период)

8.1 Реализация SEIR

Создайте `src/sir_model_SEIR.jl`. Основные изменения:

- Добавлено состояние `:E`
- Параметр σ – интенсивность перехода $E \rightarrow I$
- Массив `Ea` для временного ряда
- Новые функции обновления `exposed_update!` и `infectious_update!`

Ключевой фрагмент функции `live`:

```
# После заражения (контакт с I и успех)
individual.status = :E
exposed_update!(env, m) # S уменьшается, E увеличивается
@yield timeout(env, rand(Exponential(1/m.σ)))
individual.status = :I
infectious_update!(env, m) # E уменьшается, I увеличивается
@yield timeout(env, rand(Exponential(1/m.γ)))
individual.status = :R
recovery_update!(env, m)
```

8.2 Запуск и сравнение

```
using Random, StatsPlots
include("src/sir_model_SEIR.jl")

u0 = [990, 10, 0, 0] # S, E, I, R
p = [0.05, 10.0, 0.25, 1.0] #  $\beta$ ,  $c$ ,  $\gamma$ ,  $\sigma$ 
tmax = 40.0

Random.seed!(1234)
m = MakeSEIRModel(u0, p)
activate(m)
sir_run(m, tmax)
data = out(m)

plot(data.t, [data.S data.E data.I data.R], label=["S" "E" "I" "R"],
xlabel="Время", ylabel="Численность", title="SEIR модель")
savefig("plots/seir.png")
```

 Динамика SEIR

Рис. 5 – Динамика SEIR. Латентный период сглаживает пик инфекции по сравнению с SIR.

9. Преобразование кода в литературный стиль

Для генерации документации используем пакет `Literate.jl`. Создаём литературный файл `literate/sir_model_lit.jl`, в котором код перемежается комментариями в формате Markdown. Пример начала файла:

```
# # Дискретно-событийная SIR модель на Julia
#
# ... описание ...
using ResumableFunctions, ConcurrentSim

# ## Вспомогательные функции
function increment!(a) ...
```

Затем в терминале (из корня проекта) выполняем:

```
bash julia --project=. -e 'using Literate; Literate.notebook("literate/sir_model_lit.jl", "notebooks";
execute=true)' julia --project=. -e 'using Literate; Literate.quarto("literate/sir_model_lit.jl", "quarto")'
```

Полученные файлы `notebooks/sir_model_lit.ipynb` и `quarto/sir_model_lit.qmd` можно использовать для интерактивной работы и сборки отчёта. Для генерации HTML из Quarto:

```
bash quarto render quarto/sir_model_lit.qmd
```

10. Заключение

В ходе работы:

Реализована стохастическая дискретно-событийная SIR-модель на Julia с пакетом ConcurrentSim. Проведён анализ чувствительности к параметрам β , c , γ . Сравнены экспоненциальное и фиксированное время болезни. Выполнена оценка производительности для $N=10\,000$. Модель расширена демографическими событиями (рождение/смерть), вакцинацией и латентным периодом (SEIR). Исходный код преобразован в литературную программу, сгенерированы Jupyter Notebook и Quarto-документ. Модель готова к использованию в более сложных эпидемиологических сценариях (гетерогенные контакты, возрастные группы, календари вакцинации).